

Sequence Alignment

Via DP

Sequence Alignment

- ◆ Say, we are given a bunch of words.
 - ◆ How similar are two words?

Sequence Alignment

- ◆ Say, we are given a bunch of words.
 - ◆ How similar are two words?

occurrence

occurrence

Sequence Alignment

- ◆ Say, we are given a bunch of words.
 - ◆ How similar are two words?

occurrence

O C C U R R E N C E

ocurrance

O C U R R A N C E

Sequence Alignment

- ◆ Say, we are given a bunch of words.
 - ◆ How similar are two words?

occurrence

O C C U R R E N C E

ocurrance

O C U R R A N C E

Edit Distance

- ◆ Levenshtein 1966, Needleman-Wunsch 1970
- ◆ Needleman-Wunsch Score = 'cost' of mismatches + 'cost' of gap (for a specific perfect alignment)

Edit Distance

- ◆ Levenshtein 1966, Needleman-Wunsch 1970
- ◆ Needleman-Wunsch Score = 'cost' of mismatches + 'cost' of gap (for a specific perfect alignment)

Applications :

Basis for The Unix diff / autocorrect in editors

Search Engines (autosuggestions)

Computational Biology ('words' made up of A, G, T, C)

Edit Distance - Definition

- ◆ Input: Two strings $X = x_1, x_2, \dots, x_m$ and $Y = y_1, y_2, \dots, y_n$, where each
- ◆ Given a gap penalty δ , and mismatch cost (for each pair) $\alpha_{x,y}$
 - ◆ Total cost = Sum of the gap and mismatch costs

Steps of DP for String Problems (or generally)

- ◆ Define subproblems. //count the # of subproblems.
- ◆ Guess (part of the solutions). //count # of choices
- ◆ Relate subproblem solutions. //compute time + subproblems.
- ◆ Recurse + Memoize. //Time = Time/Subproblem
- ◆ Solve the original problems. //Combine subproblems. Or Itself a subproblem.

Subproblems for strings/sequences

- ◆ Suffixes : $X[i:], \forall i$
- ◆ prefixes : $X[:j], \forall j$
 - ◆ Both are linear size - good for us.

Subproblems for strings/sequences

- ◆ Suffixes : $X[i:], \forall i$
- ◆ prefixes : $X[:j], \forall j$
- ◆ Both are linear size - good for us.

Sometimes doesn't work... then we can guess in pairs

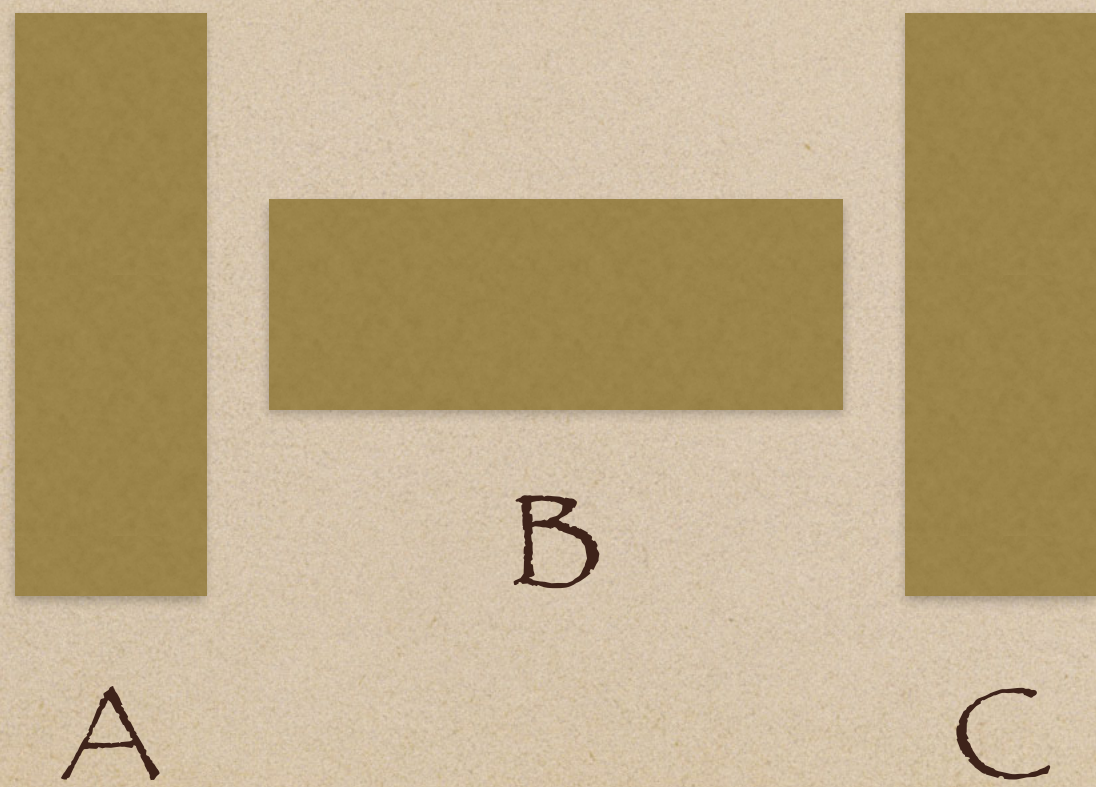
— Try all substrings! ... $X[i,j]$ — quadratic size

Parenthesization

- ◆ Optimal evaluation of an associative expression...
 $A[\emptyset], A[1], A[2], \dots, A[n - 1]$.
- ◆ Ex. - multiplying rectangular matrices.

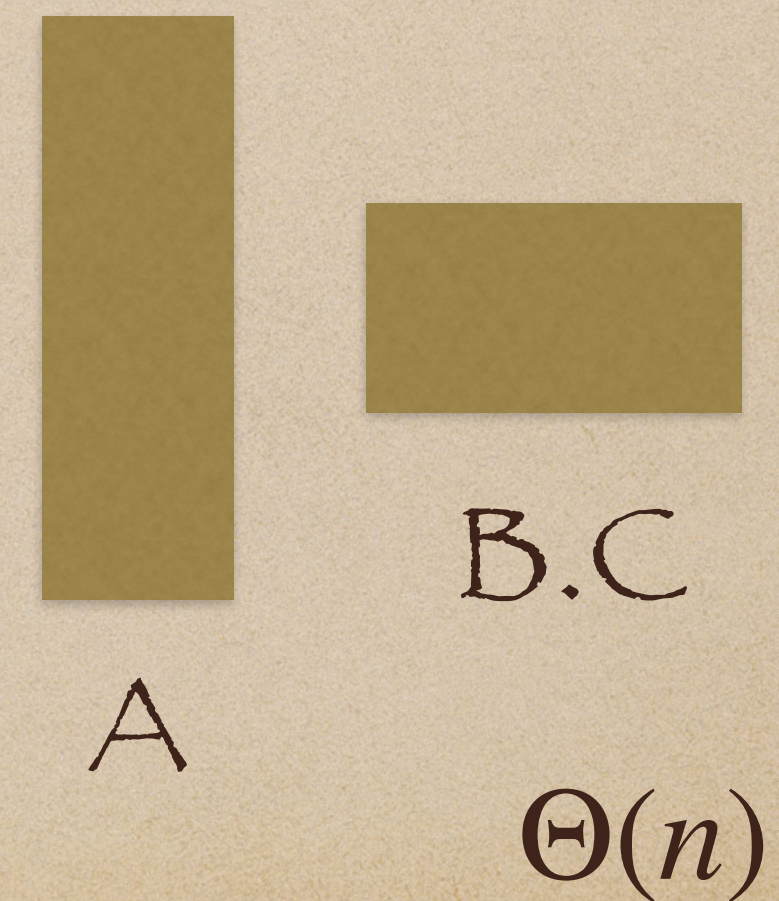
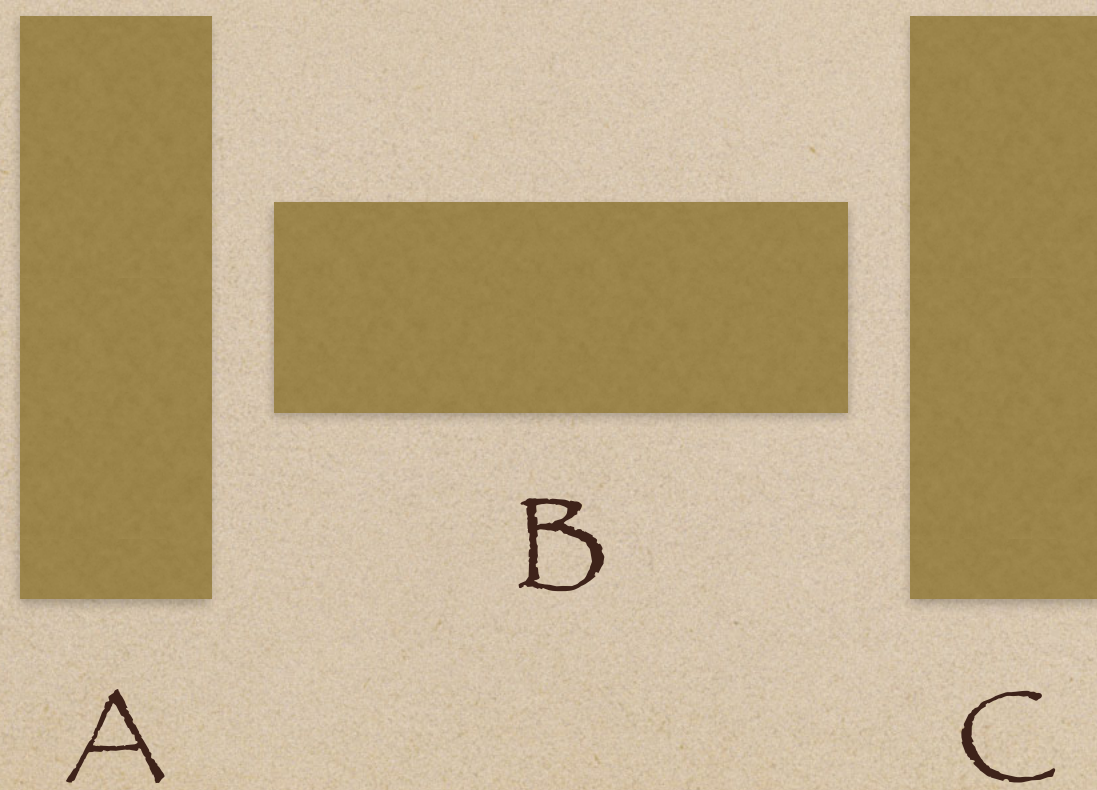
Parenthesization

- ♦ Optimal evaluation of an associative expression...
 $A[\emptyset], A[1], A[2], \dots, A[n - 1]$.
- ♦ Ex. - multiplying rectangular matrices.



Parenthesization

- ♦ Optimal evaluation of an associative expression...
 $A[\emptyset], A[1], A[2], \dots, A[n-1]$.
- ♦ Ex. - multiplying rectangular matrices.



DP steps

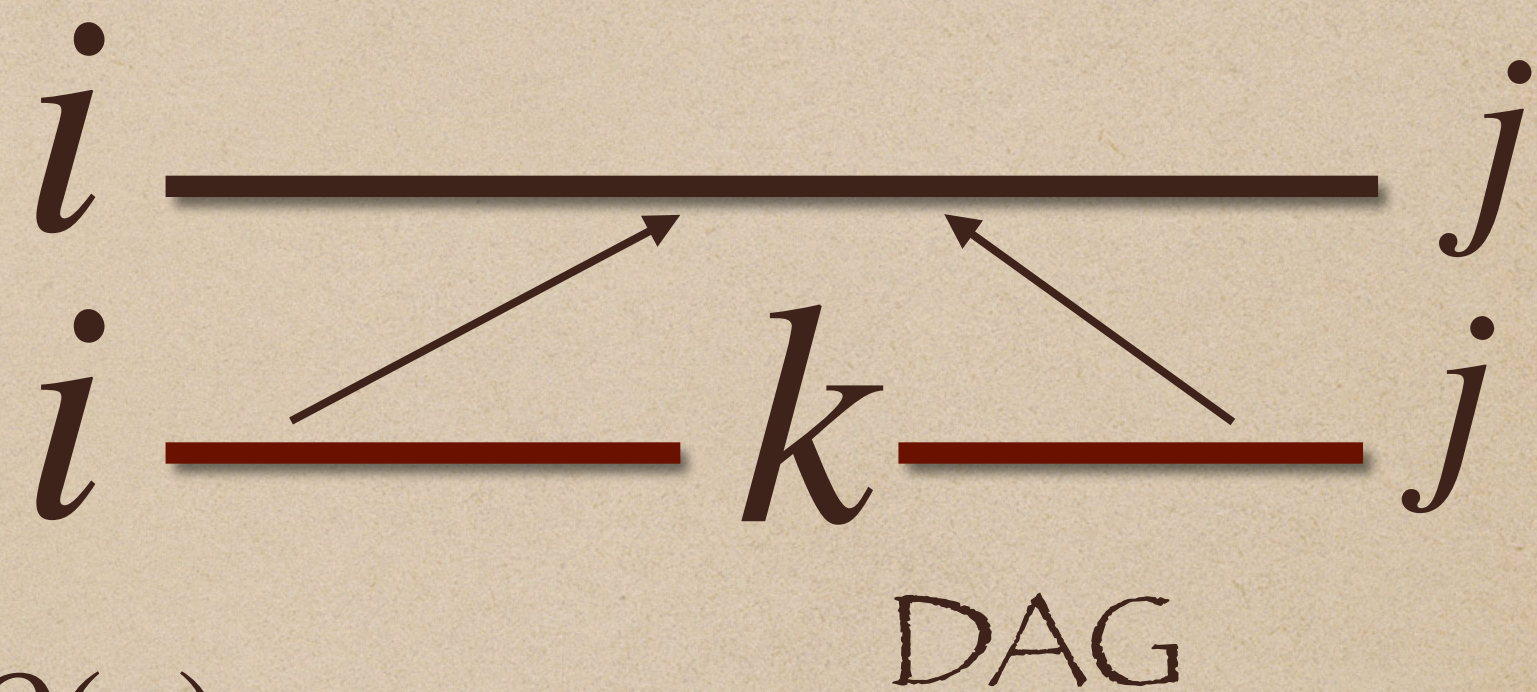
- ◆ Guessing: Outermost multiplication: (last operation that we'll do)
 - ◆ # of choices - $O(n)$
- ◆ Subproblems: Prefixes & Suffixes? - NO
 - ◆ Cost of substring: $A[i, j]$
- ◆ # of subproblems

DP steps

- ◆ Recurrence - $DP[i, j] = \min(DP[i, k] + DP[k, j] + \text{cost of multiplying } (A[i] \dots A[k-1]) \times (A[k] \dots A[j-1]));$ for k in range $(i+1, j)$

- ◆ $DP[i, i+1] = 0$

- ◆ Cost per subproblem = $O(j - i) = O(n)$



DP Steps

- ◆ Topological order: Increasing substring size - total time = $\Theta(n^3)$
- ◆ Original problem: $DP[\emptyset, n]$
 - ◆ Use parent pointers to recover parenthesis

DP Steps

- ◆ Topological order: Increasing substring size - total time = $\Theta(n^3)$
- ◆ Original problem: $DP[\emptyset, n]$
 - ◆ Use parent pointers to recover parenthesis



DP Steps

- ◆ Topological order: Increasing substring size - total time = $\Theta(n^3)$
- ◆ Original problem: $DP[\emptyset, n]$
 - ◆ Use parent pointers to recover parenthesis

Remark: This DP is not the shortest paths in the subproblem DAG!
Two dependencies $\rightarrow$ not path!

Back to Edit Distance

- ◆ Given two strings, X and Y , what is the cheapest possible sequence of characters to edit to convert X to Y .
- ◆ Operations:
 - ◆ Insert a character c
 - ◆ Delete a character c
 - ◆ Replace c with c'

Back to Edit Distance

- ◆ Given two strings, X and Y , what is the cheapest possible sequence of characters to edit to convert X to Y .

- ◆ Operations:

- ◆ Insert a character c
- ◆ Delete a character c
- ◆ Replace c with c'

If insert/delete cost 1, and replace cost $\emptyset$, minimum Edit distance is equivalent to finding the longest common subsequence.

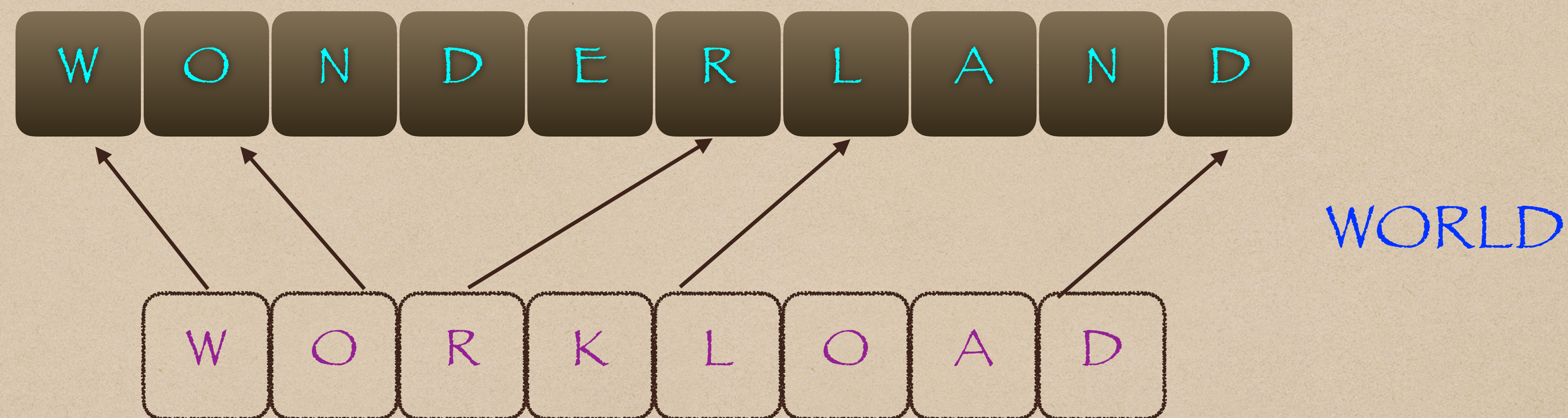
Going back to Edit Distance

W O N D E R L A N D

W O R K L O A D

Can you guess the word?

Going back to Edit Distance



Can you guess the word?

DP Route

- ◆ Subproblems for multiple strings/sequences
 - ◆ Combine suffix/prefix/substring subproblems
 - ◆ Multiple state spaces.
 - ◆ Still polynomial for $O(1)$ strings.

Edit Distance - DP

- ◆ Subproblems:

$$c(i, j) = \text{edit-distance}(x[i:], y[j:]), \quad \text{for } 0 \leq i < |x|, 0 \leq j < |y|$$

- ◆ $\Theta(|x| \cdot |y|)$ subproblems

Edit Distance - DP

- ◆ Guess whether, to turn x into y , (3 choices):
 - ◆ $x[i]$ *deleted*
 - ◆ $y[j]$ *inserted*
 - ◆ $x[i]$ *replaced by* $y[j]$

Edit Distance - DP

- ◆ Recurrence

- ◆ $c(i, j) = \text{maximum of:}$

- ◆ $\text{cost}(\text{delete } x[i]) + c(i + 1, j) \quad \text{if } i < |x|,$

- ◆ $\text{cost}(\text{insert } y[j]) + c(i, j + 1) \quad \text{if } j < |y|,$

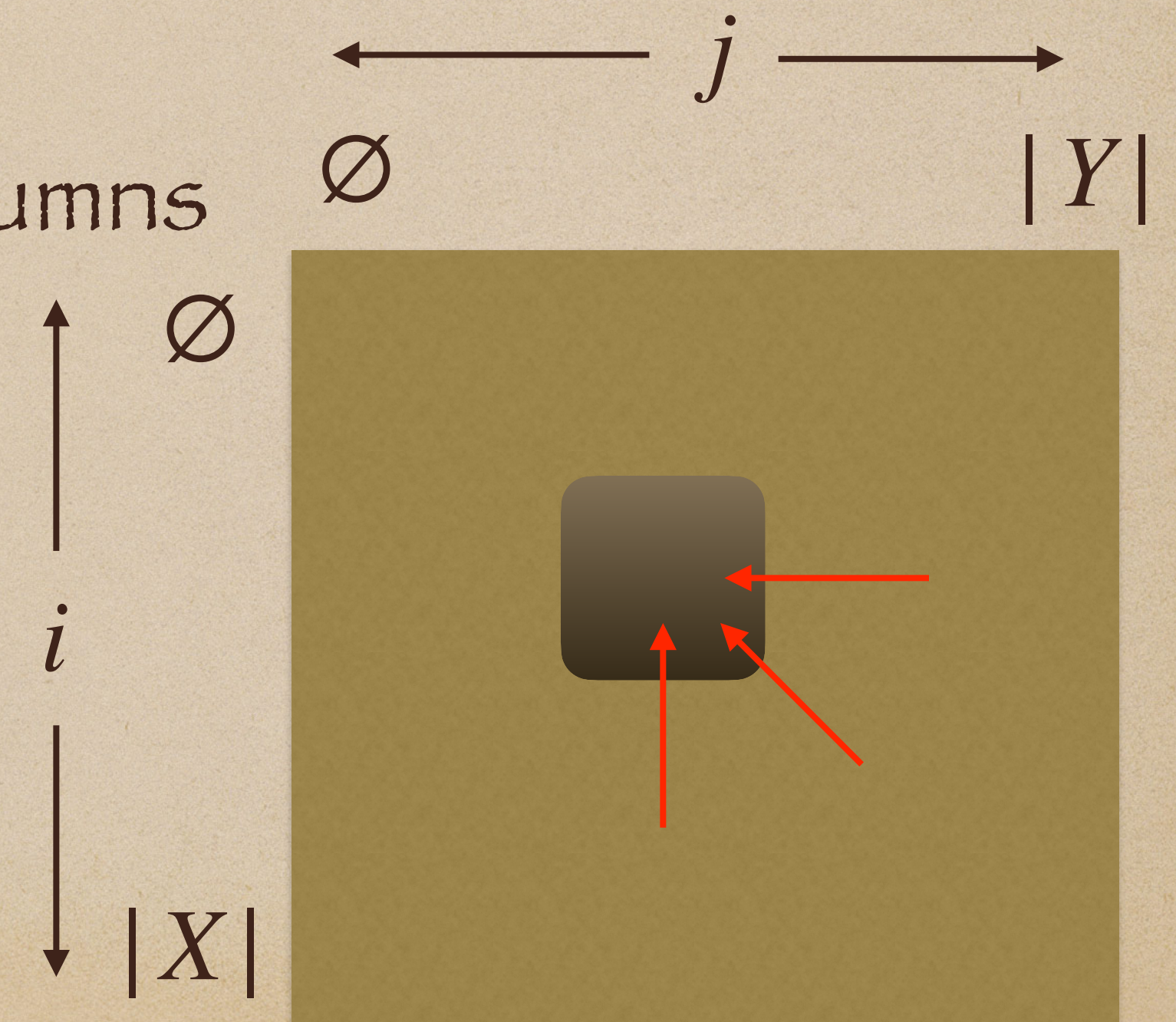
- ◆ $\text{cost}(\text{replace } x[i] \rightarrow y[j]) + c(i + 1, j + 1) \quad \text{if } i < |x| \ \& \ j < |y|$

base case : $c(|x|, |y|) = 0 \Rightarrow \Theta(1)$ time per subproblem.

Edit distance - DP

- ◆ Topological order: DAG in 2D table
 - ◆ Bottom-up or right to left
 - ◆ Only used to keep last two rows or columns

Linear space: Total time $\approx \Theta(|X| \cdot |Y|)$



Edit distance - DP

- ◆ Topological order: DAG in 2D table
 - ◆ Bottom-up or right to left
 - ◆ Only used to keep last two rows or columns

Linear space: Total time $\approx \Theta(|X| \cdot |Y|)$

Final step - $DP(\emptyset, \emptyset)$: $O(1)$ /subproblem

Time $\sim \Theta(|X| \cdot |Y|)$

